

Exploring Confidential Attributes in Active Directory

 medium.com/@offsecdeer/exploring-confidential-attributes-in-active-directory-2fac5f63c2af

Giulio Pierantoni

February 16, 2024

Confidential attributes are AD attributes only a privileged user can read, while very important data is stored inside them not many people talk about how they work.

The AD schema has a set of flags called *searchFlags* that can change some of the properties of an attribute, one of these marks them as confidential. This is used to store LAPS passwords, BitLocker keys, Unix user credentials, and other sensitive data in specific attributes that normal users shouldn't access.

For a user to read a confidential attribute they must possess the `RIGHT_DS_CONTROL_ACCESS` permission on the object containing the attribute, for example if user A has `RIGHT_DS_CONTROL_ACCESS` on *Computer* type objects they will be able to read LAPS passwords.

I thought this was a very interesting feature so I decided to dig a little in order to find out how they work. This blog post shows what I have learned in my little experiment, the DirSync "attack" is also briefly covered since it can give attackers access to confidential attributes.

Creating Confidential Attributes

To better explore this feature I am going to create a new confidential attribute, even if there are plenty by default I have fun discovering how to implement the features we exploit.

Every object in AD is of a specific **class** which defines exactly which attributes it supports. The structure of classes and attributes is stored in the Active Directory Schema, so if we want to create a custom attribute we must make a change to the schema.

First we register `SchMgmt.dll` to add the Schema manager snap-in to *mmc*, as it isn't enabled by default:

```
regsvr32 SchMgmt.dll
```

Then run *mmc.smc* and go to *add/remove snap-in*, select *Active Directory Schema*. Right click on *Attributes* -> *Create new attribute*, pick a name and description (i prefix mine with a *z* so it's easy to find at the bottom of the list):

Create New Attribute X

 Create a New Attribute Object

Identification

Common Name:

LDAP Display Name:

Unique X500 Object ID:

Description:

Syntax and Range

Syntax:

Minimum:

Maximum:

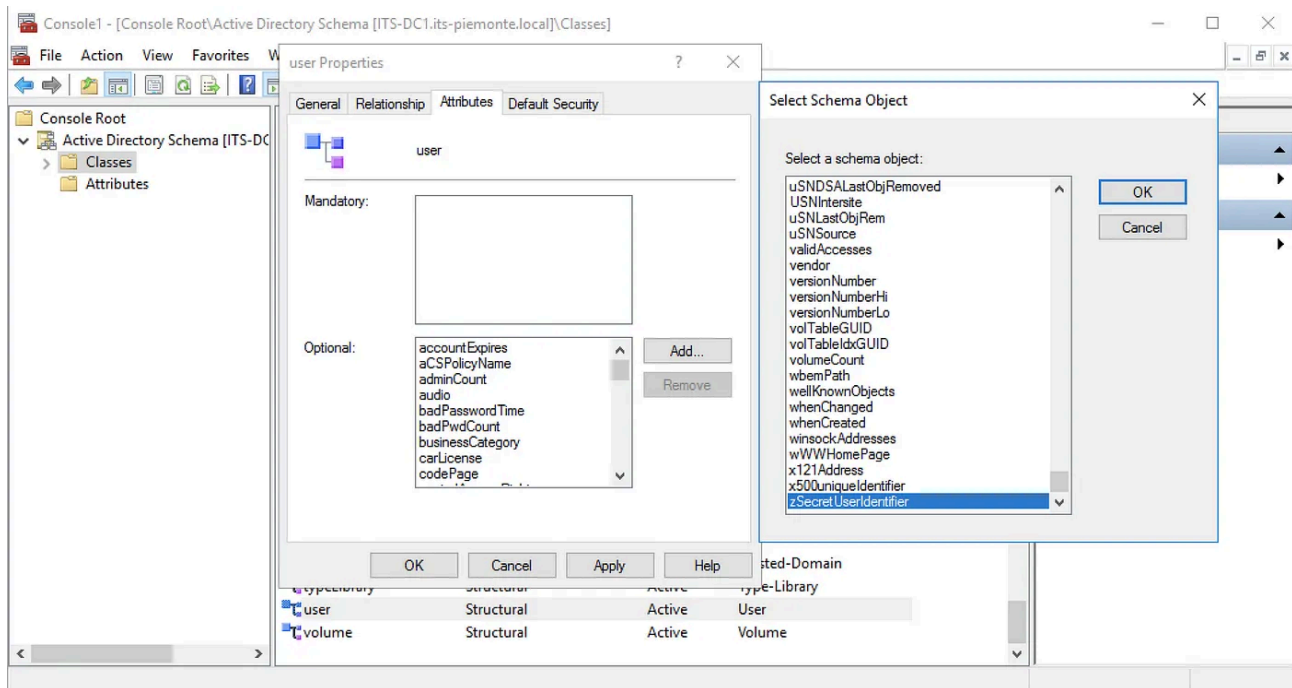
☐ Multi-Valued

Syntax defines the data type of the attribute value, in this case I picked Unicode string because I want to store text in it. Multi-Valued is for attributes that contain more than one value much like an array, like *memberOf*. The unique X500 OID must be generated by a script like the following, which outputs a new OID with Microsoft's own prefix:

```
$Prefix="1.2.840.113556.1.8000.2554"
$GUID=[System.Guid]::NewGuid().ToString()
$Parts=@()
$Parts+=[UInt64]::Parse($guid.SubString(0,4),"AllowHexSpecifier")
$Parts+=[UInt64]::Parse($guid.SubString(4,4),"AllowHexSpecifier")
$Parts+=[UInt64]::Parse($guid.SubString(9,4),"AllowHexSpecifier")
$Parts+=[UInt64]::Parse($guid.SubString(14,4),"AllowHexSpecifier")
$Parts+=[UInt64]::Parse($guid.SubString(19,4),"AllowHexSpecifier")
$Parts+=[UInt64]::Parse($guid.SubString(24,6),"AllowHexSpecifier")
$Parts+=[UInt64]::Parse($guid.SubString(30,6),"AllowHexSpecifier")
$OID=[String]::Format("{0}.{1}.{2}.{3}.{4}.{5}.{6}.
{7}",$prefix,$Parts[0],$Parts[1],$Parts[2],$Parts[3],$Parts[4],$Parts[5],$Parts[6])
$oid
```

After clicking OK expand *Classes* and select the *user* class, go to *Attributes* and *Add*, my attribute is found at the bottom of the list, very conveniently:

Press enter or click to view image in full size

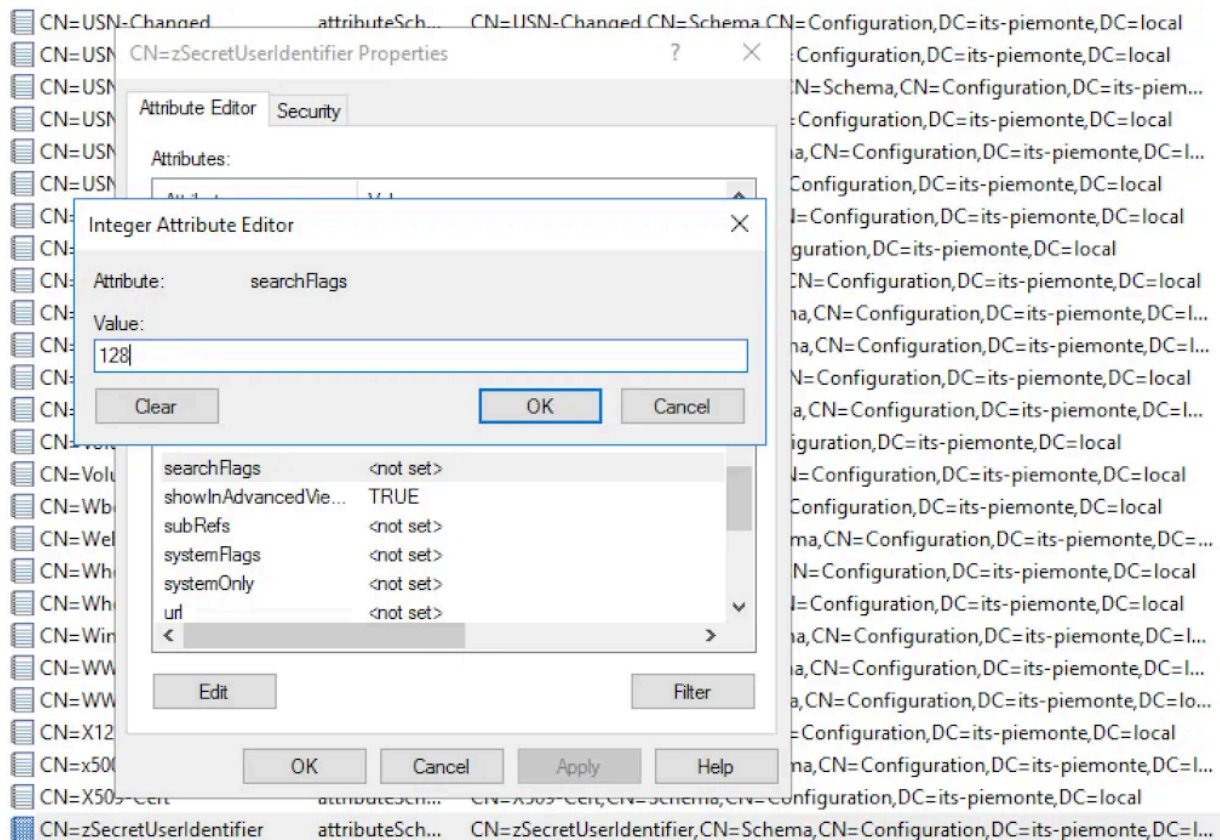


The new attribute is now part of the schema and available to *user* objects, but right now it is just like any other, we haven't made it confidential yet. To do that, we are going to have to change a specific attribute in the schema object of our attribute, because since everything in AD is described by objects, AD attributes themselves have their own objects. We can access the schema using ADSI Edit: click on *Actions* -> *Connect To* -> select *Schema* as a well known Naming Context.

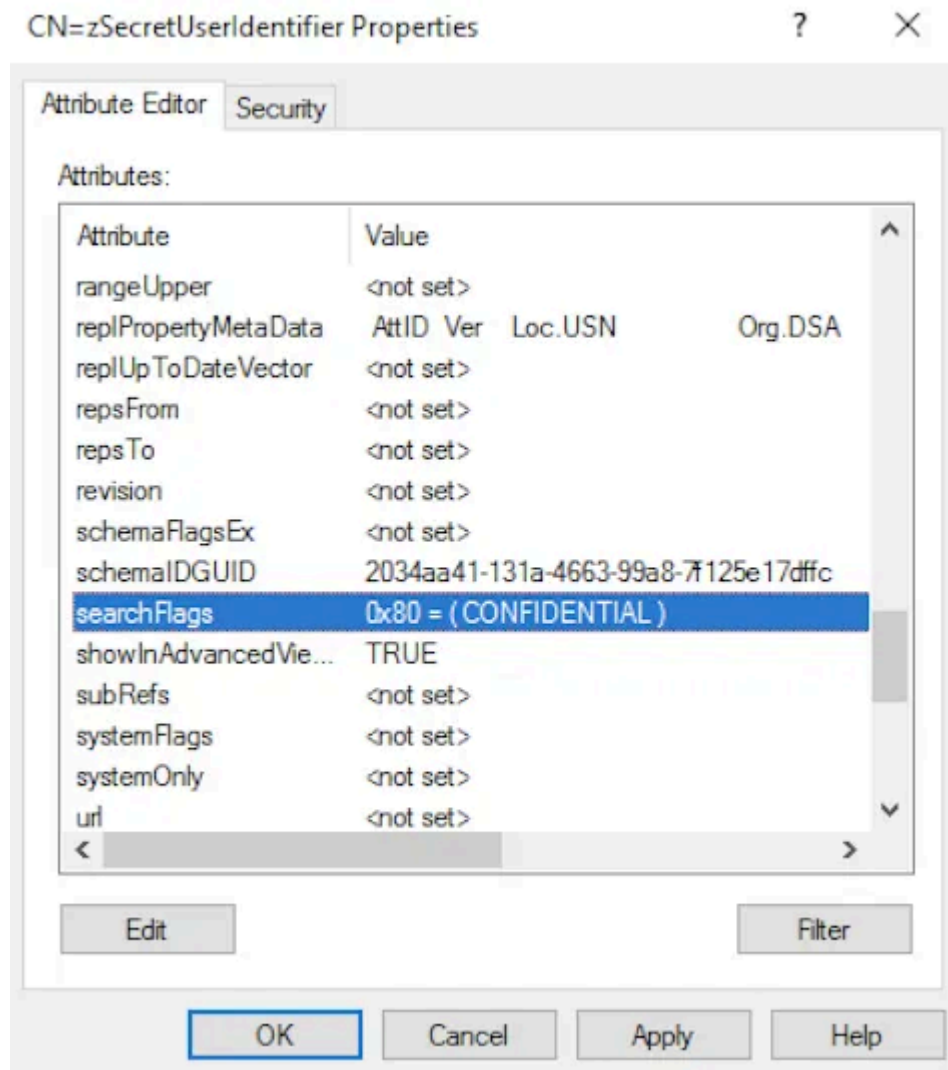
After clicking OK we'll be connected to the schema, where we are going to need to look for the distinguished name of our attribute, again conveniently found at the bottom of the attributes list. Double click on it, and pick the *Attributes* tab. We are now in front of the list of attributes describing our newly created attribute object.

In the attributes list scroll down until we run into [searchFlags](#), which we are going to set to 128 to enable the CONFIDENTIAL flag (bit 7):

Press enter or click to view image in full size



`searchFlags` is a bitmask containing a few other settings, they have to do with ambiguous name resolution ([ANR](#)), indexing, and whether the attribute is part of the filtered attribute set ([FAS](#)), which is the list of attributes that are never to be replicated to a RODC. By default the attribute is 0, meaning all the options are disabled, so all attributes are not seen as confidential and can be replicated to RODCs. Click OK and you'll see the attribute is now marked as confidential:



Confidential attributes can still be accessed, but only by security principals who have `RIGHT_DS_READ_PROPERTY` and `RIGHT_DS_CONTROL_ACCESS` permission on the specific attribute. By default domain and enterprise admins are allowed to access all confidential attributes, other users and groups can be assigned `RIGHT_DS_CONTROL_ACCESS` on one or more attributes so that they may read and modify them. Up next we are going to give a user permission to read `zSecretUserIdentifier`.

For the schema changes to take effect Active Directory Domain Services must be restarted, then we'll find `zSecretUserIdentifier` in the attribute editor tab of any user we pick from ADUC, from where we can modify its value with a double click:

Press enter or click to view image in full size

```
PS C:\Users\Administrator> Get-ADUser aarmani -Properties zSecretUserIdentifier

DistinguishedName      : CN=Armando Armani,CN=Users,DC=its-piemonte,DC=local
Enabled                 : True
GivenName               : Armando
Name                   : Armando Armani
ObjectClass             : user
ObjectGUID              : 9b92271c-d181-49ed-91fa-a8570aa1601b
SamAccountName          : aarmani
SID                     : S-1-5-21-3543871144-676301019-4006120668-1110
Surname                 : Armani
UserPrincipalName       : aarmani@its-piemonte.local
zSecretUserIdentifier   : MegaPassword!1
```

Now if we wish to delegate other non privileged principals

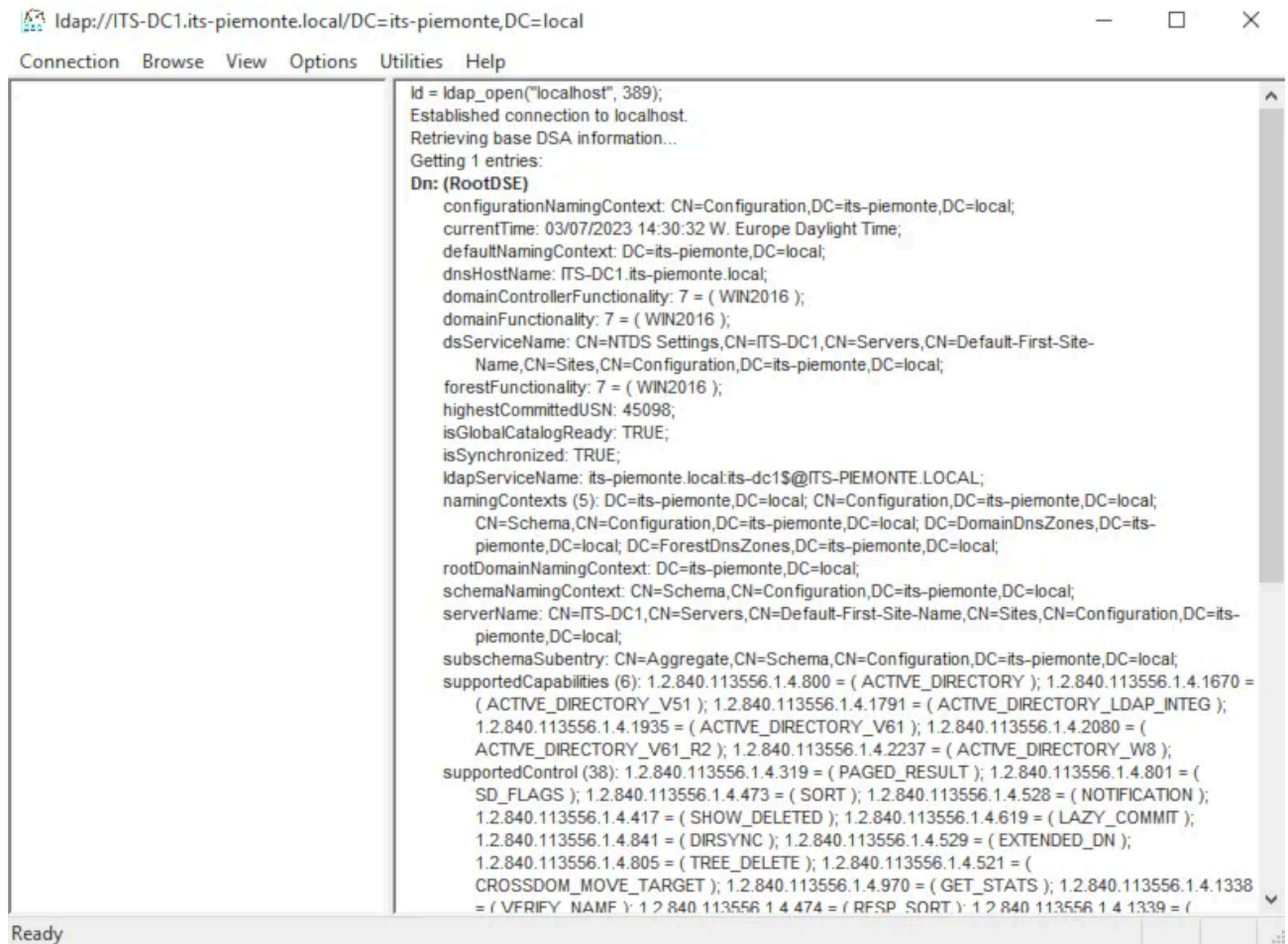
RIGHT_DS_CONTROL_ACCESS permissions to a resource we have two ways out of the box:

- GUI: ldp.exe
- cmd: dscls.exe

ldp.exe is an LDAP client with a very useful feature to modify an object's security descriptor, it comes builtin in Windows Server. From the ldp.exe main window click *Connection -> Connect*, and type the name or address of the DC.

The output should show how the client connected to the LDAP server's RootDSE:

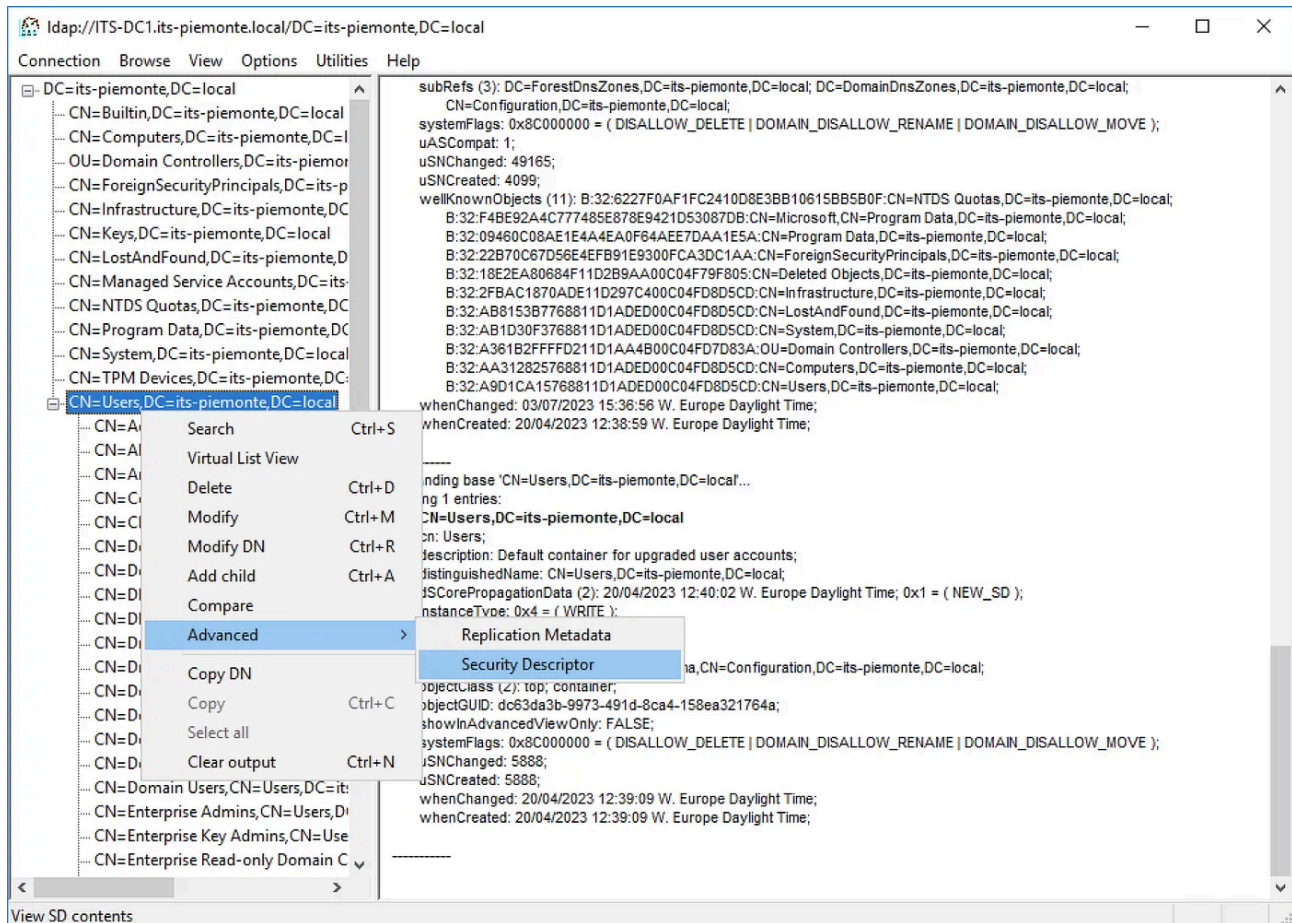
Press enter or click to view image in full size



To fully connect using our domain admin account go to *Connection -> Bind*, and *Bind as currently logged on user*:

Now to actually see the hierarchy of LDAP objects go to *View -> Tree*, select the DN of the domain root as *BaseDN*. The LDAP tree can now be explored, go pick the OU that contains the users who will make use of the attribute, like the default *Users* OU, right click on it -> *Advanced -> Security Descriptor*:

Press enter or click to view image in full size



A window asking for a DN will open, we can tick the second box to view a textual representation of the object's security descriptor if we are curious to see what the structure looks like, which is interesting to learn:

Security Descriptor:
Security Descriptor:SD Revision: 1
SD Control: 0x8c04
SE_DACL_PRESENT
SE_DACL_AUTO_INHERITED
SE_SACL_AUTO_INHERITED
SE_SELF_RELATIVE
Owner: ITS-PIEMONTE\Domain Admins [S-1-5-21-3543871144-676301019-4006120668-512]
Group: ITS-PIEMONTE\Domain Admins [S-1-5-21-3543871144-676301019-4006120668-512]
DACL:
Revision 4
Size: 1604 bytes
Aces: 33
Ace[0]
Ace Type: 0x0 - ACCESS_ALLOWED_ACE_TYPE
Ace Size: 20 bytes
Ace Flags: 0x0
Ace Mask: 0x000f01ff
DELETE
READ_CONTROL
WRITE_DAC
WRITE_OWNER
ACTRL_DS_CREATE_CHILD
ACTRL_DS_DELETE_CHILD
ACTRL_DS_LIST
ACTRL_DS_SELF
ACTRL_DS_READ_PROP
ACTRL_DS_WRITE_PROP
ACTRL_DS_DELETE_TREE
ACTRL_DS_LIST_OBJECT
ACTRL_DS_CONTROL_ACCESS
Ace Sid: NT AUTHORITY\SYSTEM [S-1-5-18]

The SACL box on the left on the other hand allows us to view the security access control list of this object, so the ACL detailing what operations will be audited in the Windows logs. We are not interested in the SACL so leave both boxes unchecked and click OK, the security descriptor editor opens:

Security descriptor - CN=Users,DC=its-piemonte,DC=local

Owner: ITS-PIEMONTE\Domain Admins

Group: ITS-PIEMONTE\Domain Admins

SD control

☒ SELF_RELATIVE	☒ DACL_PRESENT	☐ SACL_PRESENT
☐ OWNER_DEFAULTED	☐ DACL_PROTECTED	☐ SACL_PROTECTED
☐ GROUP_DEFAULTED	☒ DACL_AUTO_INHERITED	☒ SACL_AUTO_INHERITED
	☐ DACL_DEFAULTED	☐ SACL_DEFAULTED

DACL (33 ACEs)

Type	Trustee	Rights	Flags
Allow	NT AUTHORITY\SYSTEM	Full control	
Allow	ITS-PIEMONTE\Domain Admins	Write, List object, Write DACL, Write...	
Allow	BUILTIN\Account Operators	Create child, Delete child (user)	
Allow	BUILTIN\Account Operators	Create child, Delete child (group)	
Allow	BUILTIN\Print Operators	Create child, Delete child (printQueue)	
Allow	NT AUTHORITY\Authenticated Users	Read	
Allow	BUILTIN\Account Operators	Create child, Delete child (printQueue)	

SACL

Type	Trustee	Rights	Flags
------	---------	--------	-------

Update

☐ Owner ☐ Group ☐ DACL ☐ SACL

Update Close

We are interested in adding a new ACE to the ACL, so click *Add* :

ACE - Access Control Entry

Trustee: aarmani

ACE type: ☒ Allow ☐ Deny ☐ Audit ☐ Alarm

Access mask

☐ Read property	☐ Write property	☐ Create child	☒ Control access
☐ List	☐ Write DACL	☐ Delete child	☐ Extended write
☐ List object	☐ Write owner	☐ Delete	
☒ Read permissions	☐ Write SACL	☐ Delete tree	

ACE flags

☒ Inherit	☐ Inherited	☐ Success
☐ No propagate	☐ Inherit only	☐ Failure

Object type: zSecretUserIdentifier - attribute

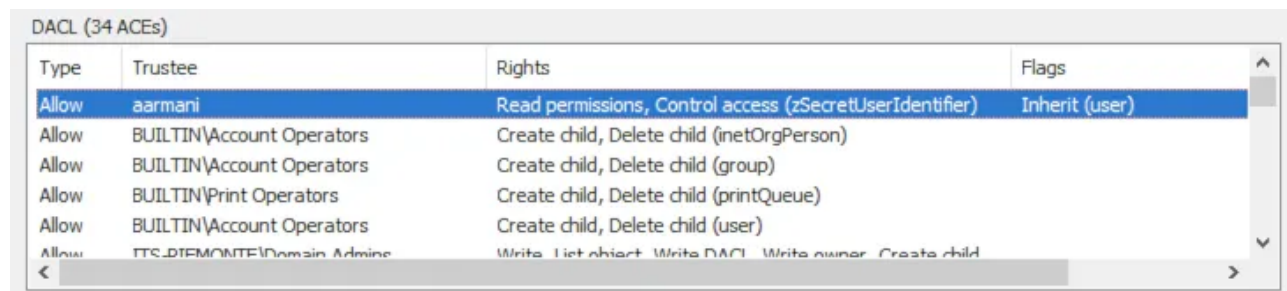
Inherited object type: user

OK Cancel

- *Trustee* should contain the SamAccountName of the security principal to be delegated
- *Access mask* should have **Read permissions** and **Control access checked (write property)** too if we want the users to modify the attributes and not just read them)
- *ACE flags* should have **Inherit** checked
- *Object type* is set to the confidential attribute the principal will be able to read
- *Inherited object type* is set to **user**

Click on OK, the DACL should now appear as such:

Press enter or click to view image in full size



Type	Trustee	Rights	Flags
Allow	aarmani	Read permissions, Control access (zSecretUserIdentifier)	Inherit (user)
Allow	BUILTIN\Account Operators	Create child, Delete child (inetOrgPerson)	
Allow	BUILTIN\Account Operators	Create child, Delete child (group)	
Allow	BUILTIN\Print Operators	Create child, Delete child (printQueue)	
Allow	BUILTIN\Account Operators	Create child, Delete child (user)	
Allow	ITS-DIFMONTE\Domain Admins	Write, List object, Write DACL, Write owner, Create child	

Then click on *Update* making sure the *DACL* option next to it is checked, the other boxes should stay unchecked unless you made other modifications to the security descriptor.

Now if we want we can do a text dump of the OU's SD to see the textual representation of the new ACE:

```

Security Descriptor:
Security Descriptor:SD Revision: 1
SD Control: 0x8c04
    SE_DACL_PRESENT
    SE_DACL_AUTO_INHERITED
    SE_SACL_AUTO_INHERITED
    SE_SELF_RELATIVE
Owner: ITS-PIEMONTE\Domain Admins [S-1-5-21-3543871144-676301019-4006120668-512]
Group: ITS-PIEMONTE\Domain Admins [S-1-5-21-3543871144-676301019-4006120668-512]
DACL:
    Revision    4
    Size:       1676 bytes
    # Aces:     34
    Ace[0]
        Ace Type: 0x5 - ACCESS_ALLOWED_OBJECT_ACE_TYPE
        Ace Size: 72 bytes
        Ace Flags: 0x2
            CONTAINER_INHERIT_ACE
        Object Ace Mask: 0x00020100
            READ_CONTROL
            ACTRL_DS_CONTROL_ACCESS
        Object Ace Flags: 0x3
            ACE_OBJECT_TYPE_PRESENT
            ACE_INHERITED_OBJECT_TYPE_PRESENT
        Object Ace Type: zSecretUserIdentifier - 2034aa41-131a-4663-99a8-7f125e17dff
        Inherited object type: user - bf967aba-0de6-11d0-a285-00aa003049e2
        Object Ace Sid: ITS-PIEMONTE\aarmani [S-1-5-21-3543871144-676301019-4006120668-1110]
    Ace[1]
        Ace Type: 0x5 - ACCESS_ALLOWED_OBJECT_ACE_TYPE
        Ace Size: 44 bytes
        Ace Flags: 0x0
        Object Ace Mask: 0x00000003
            ACTRL_DS_CREATE_CHILD
            ACTRL_DS_DELETE_CHILD
        Object Ace Flags: 0x1
            ACE_OBJECT_TYPE_PRESENT
        Object Ace Type: inetOrgPerson - 4828cc14-1437-45bc-9b07-ad6f015e5f28
        Object Ace Sid: BUILTIN\Account Operators [S-1-5-32-548]

```

Time to check these new permissions. Here's a normal user account, *tantani*, trying to read the attribute of its own account:

Press enter or click to view image in full size

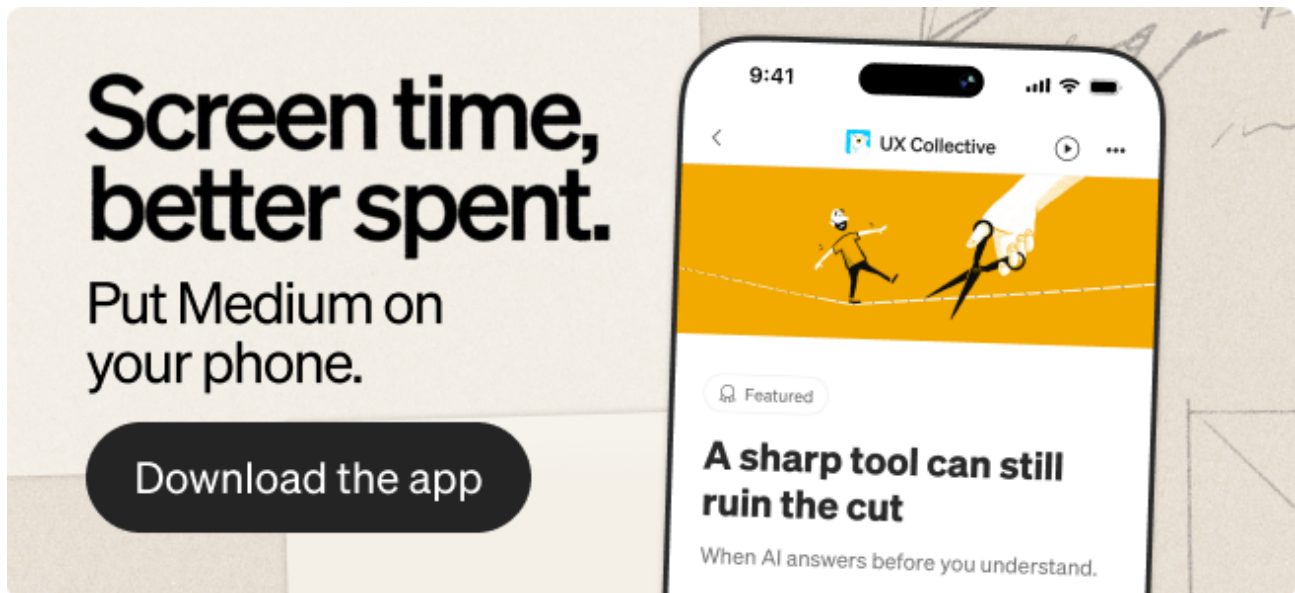
```

*Evil-WinRM* PS C:\Users\tantani\Documents> whoami
its-piemonte\tantani
*Evil-WinRM* PS C:\Users\tantani\Documents> Get-ADUser tantani -Properties zSecretUserIdentifier

DistinguishedName : CN=Tizio Antani,CN=Users,DC=its-piemonte,DC=local
Enabled           : True
GivenName        : Tizio
Name             : Tizio Antani
ObjectClass       : user
ObjectGUID       : 08211005-23b8-4510-9d37-0527df206718
SamAccountName    : tantani
SID              : S-1-5-21-3543871144-676301019-4006120668-1105
Surname          : Antani
UserPrincipalName : tantani@its-piemonte.local

```

The attribute does not appear at all in the output, if it was unset and the user had the required permissions to access it then the attribute name would appear, with an empty line after the colon.



On the other hand here is the delegated user, aarmani, trying to read the same user's attribute:

Press enter or click to view image in full size

```
*Evil-WinRM* PS C:\Users\aarmani\Documents> whoami
its-piemonte\aarmani
*Evil-WinRM* PS C:\Users\aarmani\Documents> Get-ADUser tantani -Properties zSecretUserIdentifier

DistinguishedName      : CN=Tizio Antani,CN=Users,DC=its-piemonte,DC=local
Enabled                 : True
GivenName               : Tizio
Name                    : Tizio Antani
ObjectClass              : user
ObjectGUID              : 08211005-23b8-4510-9d37-0527df206718
SamAccountName          : tantani
SID                     : S-1-5-21-3543871144-676301019-4006120668-1105
Surname                 : Antani
UserPrincipalName       : tantani@its-piemonte.local
zSecretUserIdentifier   : 0wO_nothere!
```

We have successfully added a confidential attribute in our AD!

Enumerating Confidential Attributes

If we want to query the AD schema to look for all attributes that are marked as confidential we can use an LDAP filter with the *Get-ADObject* cmdlet by setting the schema's distinguished name as the baseDN.

To do that first we find the *SchemaNamingContext* with *Get-ADRootDSE*:

```

PS C:\Users\Administrator> Get-ADRootDSE

configurationNamingContext : CN=Configuration,DC=its-piemonte,DC=local
currentTime                : 03/07/2023 17:48:40
defaultNamingContext        : DC=its-piemonte,DC=local
dnsHostName                 : ITS-DC1.its-piemonte.local
domainControllerFunctionality : Windows2016
domainFunctionality         : Windows2016Domain
dsServiceName               : CN=NTDS Settings,CN=ITS-DC1,CN=Servers,CN=Default-First-Site-Name,CN=Sites,CN=Configura
                             tion,DC=its-piemonte,DC=local
forestFunctionality         : Windows2016Forest
highestCommittedUSN         : 49188
isGlobalCatalogReady       : {TRUE}
isSynchronized              : {TRUE}
ldapServiceName             : its-piemonte.local:its-dc1$@ITS-PIEMONTE.LOCAL
namingContexts              : {DC=its-piemonte,DC=local, CN=Configuration,DC=its-piemonte,DC=local,
                             CN=Schema,CN=Configuration,DC=its-piemonte,DC=local,
                             DC=DomainDnsZones,DC=its-piemonte,DC=local...}
rootDomainNamingContext     : DC=its-piemonte,DC=local
schemaNamingContext         : CN=Schema,CN=Configuration,DC=its-piemonte,DC=local
serverName                  : CN=ITS-DC1,CN=Servers,CN=Default-First-Site-Name,CN=Sites,CN=Configuration,DC=its-piemonte,DC=local
subschemaSubentry           : CN=Aggregate,CN=Schema,CN=Configuration,DC=its-piemonte,DC=local
supportedCapabilities        : {1.2.840.113556.1.4.800 (LDAP_CAP_ACTIVE_DIRECTORY_OID), 1.2.840.113556.1.4.1670
                             (LDAP_CAP_ACTIVE_DIRECTORY_V51_OID), 1.2.840.113556.1.4.1791
                             (LDAP_CAP_ACTIVE_DIRECTORY_LDAP_INTEG_OID), 1.2.840.113556.1.4.1935
                             (LDAP_CAP_ACTIVE_DIRECTORY_V61_OID)...}
supportedControl             : {1.2.840.113556.1.4.319 (LDAP_PAGED_RESULT_OID_STRING), 1.2.840.113556.1.4.801
                             (LDAP_SERVER_SD_FLAGS_OID), 1.2.840.113556.1.4.473 (LDAP_SERVER_SORT_OID),
                             1.2.840.113556.1.4.528 (LDAP_SERVER_NOTIFICATION_OID)...}
supportedLDAPPolicies        : {MaxPoolThreads, MaxPercentDirSyncRequests, MaxDatagramRecv, MaxReceiveBuffer...}
supportedLDAPVersion         : {3, 2}
supportedSASLMechanisms      : {GSSAPI, GSS-SPNEGO, EXTERNAL, DIGEST-MD5}

```

This is what an attribute object's own attributes look like:

```

adminDescription            : Backup passwords for user account login
adminDisplayName            : zSecretUserIdentifier
attributeID                 : 1.2.840.113556.1.8000.2554.27241.12540.59496.18081.41372.638243.4766210
attributeSyntax              : 2.5.5.12
canonicalName               : its-piemonte.local/Configuration/Schema/zSecretUserIdentifier
CN                          : zSecretUserIdentifier
Created                     : 03/07/2023 10:18:57
createTimeStamp              : 03/07/2023 10:18:57
Deleted                     : 
Description                  : 
displayName                  : 
distinguishedName           : CN=zSecretUserIdentifier,CN=Schema,CN=Configuration,DC=its-piemonte,DC=local
dsCorePropagationData       : {01/01/1601 01:00:00}
instanceType                : 4
isDeleted                   : 
isSingleValued              : True
lastKnownParent             : 
LDAPDisplayName             : zSecretUserIdentifier
modified                    : 03/07/2023 10:37:52
modifyTimeStamp              : 03/07/2023 10:37:52
msDS-IntId                  : -1831205341
Name                        : zSecretUserIdentifier
ntSecurityDescriptor         : System.DirectoryServices.ActiveDirectorySecurity
objectCategory              : CN=AttributeSchema,CN=Schema,CN=Configuration,DC=its-piemonte,DC=local
objectClass                 : attributeSchema
objectGUID                  : 549d13a6-e9e7-40c9-acf7-ca0c518f3360
oMSyntax                    : 64
protectedFromAccidentalDeletion : False
schemaIDGUID                : {65, 170, 52, 32...}
sDRightsEffective           : 15
searchFlags                 : 128
showInAdvancedViewOnly      : True
uSNChanged                  : 40992
uSNCreated                  : 40988
whenChanged                 : 03/07/2023 10:37:52
whenCreated                 : 03/07/2023 10:18:57

```

The attributes we may be interested in are: DistinguishedName, Name, adminDescription, and searchFlags, this last one actually containing the CONFIDENTIAL flag so it will be used within the filter.

Microsoft provides an [OID](#) specifically for bitwise AND checks on flags:

1.2.840.113556.1.4.803, this will be used to filter out all attributes that do not have a *searchFlags* attribute with the CONFIDENTIAL bit set.

The resulting command to obtain all confidential attributes is:

```

Get-ADObject -LDAPFilter "(&(objectClass=attributeSchema)
(searchFlags:1.2.840.113556.1.4.803:=128))" -SearchBase (Get-

```

```
ADRootDSE).SchemaNamingContext -Properties * | select  
adminDisplayName,adminDescription | fl
```

This is useful because any domain user can query the schema, so users don't need to be able to read the attribute's values in order to check which confidential attributes exist in the schema:

```
adminDisplayName : ms-DS-Transformation-Rules-Compiled  
adminDescription : Blob containing compiled transformation rules.  
  
adminDisplayName : ms-DS-IssuerCertificates  
adminDescription : The keys used to sign certificates issued by the Registration Service.  
  
adminDisplayName : zSecretUserIdentifier  
adminDescription : Backup passwords for user account login
```

Now to find out what users have access to what attributes, dscls.exe can be used to read any object's DACL, so we can specify the DN of the *Users* OU and get all of the ACEs in its DACL:

Press enter or click to view image in full size

Allow BUILTIN\Account Operators	SPECIAL ACCESS for user CREATE CHILD DELETE CHILD
Allow ITS-PIEMONTE\aarmani	SPECIAL ACCESS for zSecretUserIdentifier READ PERMISSIONS CONTROL ACCESS
Allow ITS-PIEMONTE\Key Admins	SPECIAL ACCESS for msDS-KeyCredentialLink <Inherited from parent> WRITE PROPERTY READ PROPERTY

So with a bit of PowerShell we can just obtain the list of principals who are given permissions explicitly to the attribute, in this case *aarmani* can only read other users' *zSecretUserIdentifier*.

Automatic Enumeration: Declassify.ps1

Declassify is a very small and awfully basic PowerShell script that lists confidential attributes of user and computer objects and finds if any principals have non-default permissions. It misses quite a few things as of now, it's very WIP and I'll put more work into it when I can to make it more effective, right now it's just an experimental little thing to quickly see if there are any custom confidential attributes in the domain, and if that wasn't enough it has to be run on a DC because it relies on the output of dscls.exe, which only gets installed with AD DS on domain controllers.

List every confidential attribute:

Press enter or click to view image in full size

```
PS C:\Users\Administrator> .\desktop\Declassify.ps1 -List
[ALL CONFIDENTIAL ATTRIBUTES]
ms-FVE-RecoveryPassword
ms-FVE-KeyPackage
ms-TPM-OwnerInformation
ms-PKI-Credential-Roaming-Tokens
ms-PKI-RoamingTimeStamp
ms-PKI-DPAPIMasterKeys
ms-PKI-AccountCredentials
UnixUserPassword
ms-TPM-Owner-Information-Temp
ms-Kds-KDF-AlgorithmID
ms-Kds-KDF-Param
ms-Kds-SecretAgreement-AlgorithmID
ms-Kds-SecretAgreement-Param
ms-Kds-PublicKey-Length
ms-Kds-PrivateKey-Length
ms-Kds-RootKeyData
ms-Kds-Version
ms-Kds-DomainID
ms-Kds-UseStartTime
ms-Kds-CreateTime
ms-DS-Transformation-Rules-Compiled
ms-DS-Issuer-Certificates
zSecretUserIdentifier
PS C:\Users\Administrator>
```

Find non-standard permissions to confidential attributes:

Press enter or click to view image in full size

```
PS C:\Users\Administrator> .\desktop\Declassify.ps1
[USER ATTRIBUTES]
ITS-PIEMONTE\tantani --[READ]--> UnixUserPassword
ITS-PIEMONTE\aarmani --[READ]--> zSecretUserIdentifier
[COMPUTER ATTRIBUTES]
PS C:\Users\Administrator> █
```

So far it only finds explicit permissions, not inherited ones. For example it doesn't find users who have CONTROL ACCESS over the root of the domain and inherit it to all children objects.

This is the full script if anyone wants to play with it.

```
param ([switch]$List = $False)
$attributes = Get-ADObject -LDAPFilter "(&(objectClass=attributeSchema)
(searchFlags:1.2.840.113556.1.4.803:=128))" -SearchBase (Get-
ADRootDSE).SchemaNamingContext | foreach {$_.Name}
if ($list -eq $true)
{
    Write-Host "[ALL CONFIDENTIAL ATTRIBUTES]"
```

```

    foreach ($attr in $attributes)
    {
        Write-Host $attr
    }
    Exit
}
$domainOU = Get-ADDomain | foreach {$_ .DistinguishedName}
$usersContainer = "CN=Users," + $domainOU
$computersContainer = "CN=Computers," + $domainOU
$output = dsacls $usersContainer
Write-Host "[USER ATTRIBUTES]"
foreach ($attr in $attributes)
{
    $ace = $output | Select-String $attr
    if ($ace -like "*SPECIAL ACCESS*")
    {
        if ($ace -match 'Allow ([^\s+)\s')
        {
            Write-Host $Matches[1] "--[READ]-->"$attr
        }
    }
}
$output = dsacls $computersContainer
Write-Host "[COMPUTER ATTRIBUTES]"
foreach ($attr in $attributes)
{
    $ace = $output | Select-String $attr
    if ($ace -like "*SPECIAL ACCESS*")
    {
        if ($ace -match 'Allow ([^\s+)\s')
        {
            Write-Host $Matches[1] "--[READ]-->"$attr
        }
    }
}
}

```

DirSync

[DirSync](#) is an LDAP extended control introduced by Microsoft, it should be used by domain controllers during replication to request all the data that has changed since the last DirSync operation. The data that can be replicated with DirSync includes confidential attributes, so it is a very good way to access them. These are the permissions required to do a DirSync operation:

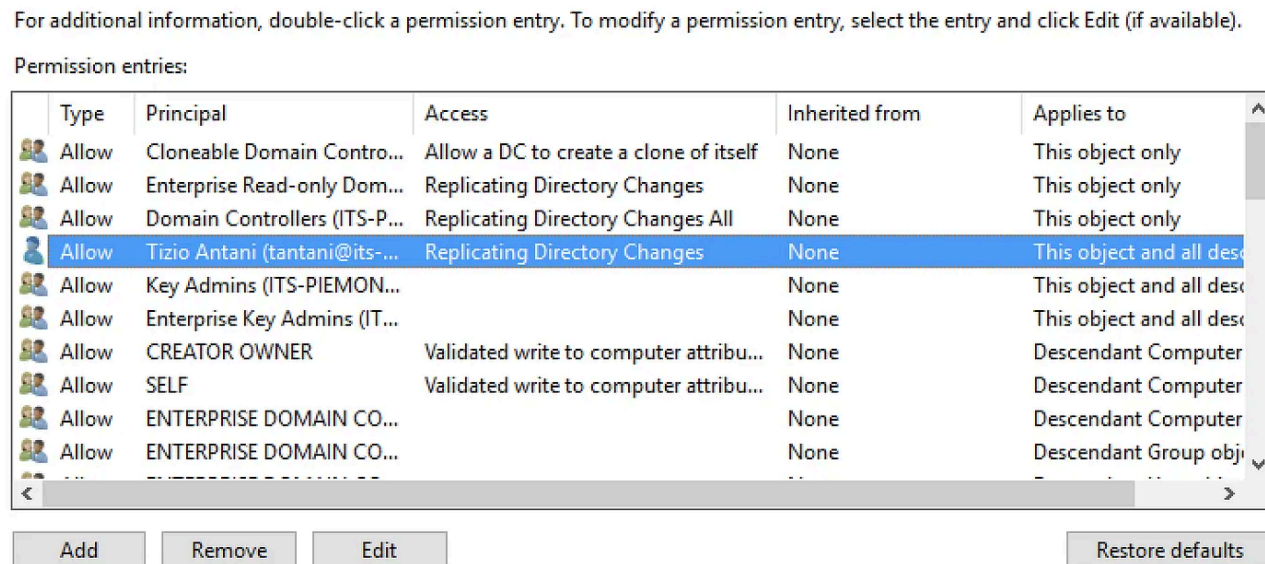
- DS-Replication-Get-Changes
- DS-Replication-Get-Changes-In-Filtered-Set

Basically the DCSync permissions minus DS-Replication-Get-Changes-All, so if you can DCSync the domain you can also DirSync it, but if a user is missing DS-Replication-Get-

Changes-All they won't be able to DCSync, so forget about grabbing those domain admin hashes.

Take the user *tantani*, they have DS-Replicaition-Get-Changes on the domain:

Press enter or click to view image in full size



Here *tantani* used the DirSync PoC linked above to successfully access the `zSecretUserIdentifier` attribute despite not having explicit permissions to read it, an access attempt with `Get-ADUser` is also tried to prove the user can't normally see the attribute, which isn't returned at all due to insufficient permissions:

Press enter or click to view image in full size

```
*Evil-WinRM* PS C:\Users\tantani\Documents> Sync-Attributes -LDAPFilter "(samaccountname=tantani)" -Attributes zSecretUserIdentifier
Object: CN=Tizio Antani,CN=Users,DC=its-piemonte,DC=local
zSecretUserIdentifier: OwO_nothere!

*Evil-WinRM* PS C:\Users\tantani\Documents> Get-ADUser tantani -Properties zSecretUserIdentifier

DistinguishedName : CN=Tizio Antani,CN=Users,DC=its-piemonte,DC=local
Enabled           : True
GivenName        : Tizio
Name             : Tizio Antani
ObjectClass      : user
ObjectGUID       : 08211005-23b8-4510-9d37-0527df206718
SamAccountName   : tantani
SID              : S-1-5-21-3543871144-676301019-4006120668-1105
Surname          : Antani
UserPrincipalName : tantani@its-piemonte.local
```

Most attributes can be retrieved as long as DS-Replication-Get-Changes is assigned to a user, DS-Replication-Get-Changes-In-Filtered-Set is useful to access attributes that are part of the FAS like LAPS passwords, found in the *ms-Mcs-AdmPwd* attribute of the computer object.

Interesting Confidential Attributes

ms-Mcs-AdmPwd

Only present in computer accounts when LAPS is enabled, it will store the clear-text password for the computer's default administrator

msDS-ManagedPassword

Only exists on objects of class *msDS-GroupManagedServiceAccount*, which are group-managed service accounts (gMSA). This attribute is a BLOB (Binary Large Object) containing the account's password (a post on gMSA may come in the future)

UnixUserPassword

It will sometimes contain the SHA1 hash of POSIX user accounts when the domain has AD-joined *nix computers. In past Windows versions this role was covered by either *userPassword* or *msSFU30Password*, so these attributes may still be present in domains that have existed on DCs running Windows 2000 or Windows Server 2003

ms-Kds-RootKeyData

This and the other attributes starting with *ms-Kds* which can be seen in the output of the script above are used by the Key Distribution Service (KDS) for the generation of gMSA passwords. While the gMSA credentials are rotated automatically (the whole point of gMSA) the attributes used to generate new passwords are not updated until an admin does it manually, so if an attacker gets access to these attributes they will be able to predict the future passwords of managed service accounts. This attack is known as [golden gMSA](#) and the gMSA passwords can be generated completely offline

ms-PKI-DPAPIMasterKeys

A user-specific symmetric key used to encrypt and decrypt [roamed credentials](#) using DPAPI, it is stored in encrypted format and its decryption requires either the user's password (or SHA1 hash) or the domain backup key, retrievable only by administrators

ms-PKI-AccountCredentials

It contains user secrets that are synced with credential roaming, including private keys, certificates, and passwords. These are decrypted with the key found in *ms-PKI-DPAPIMasterKeys*

ms-FVE-RecoveryPassword

BitLocker key required to recover a disk partition protected by Full Volume Encryption (FVE)

Conclusion

How useful is this information? Probably very little, but it helped me learn plenty new things about the way AD works under the hood and it was a fun little experiment, I'm not going to

pretend it has any operational value in a red team engagement though.

Confidential attributes are used to store information that should remain, well... confidential. For that reason the permissions required to access these should be assigned carefully and one should be mindful of the replication permissions as well, as they can be used to read every confidential attribute regardless of their permissions.

References

- <https://learn.microsoft.com/en-US/troubleshoot/windows-server/windows-security/mark-attribute-as-confidential>
- https://learn.microsoft.com/en-us/openspecs/windows_protocols/ms-adts/e6685d31-5d87-42d0-8a5f-e55d337f47cd
- <https://simondotsh.com/infosec/2022/07/11/dirsync.html>
- <https://cqureacademy.com/blog/extracting-roamed-private-keys>
- <https://www.ibm.com/docs/en/aix/7.2?topic=servers-active-directory-password-attribute-selection>